МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Центр образовательных программ топ-уровня в сфере информационных технологий Образовательного центра Института №8

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Операционные системы и архитектура компьютера»

на тему: « **Управление потоками. Многопоточная реализация программы с параллельными операциями»**

Выполнил: студент гр.

М8О-206БВ-25

Толстопятов Трофим Михайлович

Проверил: Ефимов Д.А.

Оценка:

Дата:

Москва 2026

СОДЕРЖАНИЕ

Цель работы 2
Архитектура решения 3
Описание задачи 3
Схема взаимодействия 4
Применённые системные примитивы 5
Ключевые фрагменты кода 5
Работа с потоками 6
Функция потока 7
Медианный фильтр 7
Результаты работы 8
Выводы 12
Приложения 13
Приложение А: ссылка на Git 14

Цель работы

Приобретение практических навыков в:

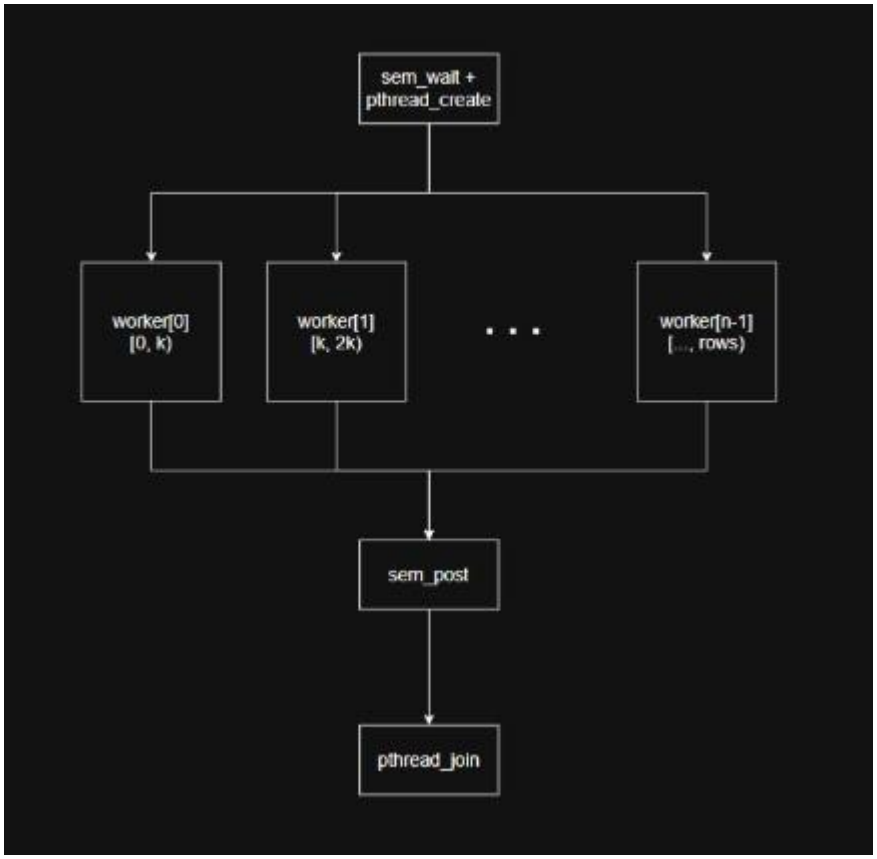
- управлении потоками в ОС Linux средствами POSIX Threads (pthread);
- обеспечении синхронизации между потоками посредством семафоров;
- обработке системных ошибок, возникающих при создании потоков и работе с разделяемой памятью.

Архитектура решения

Описание задачи

Программа получает через командную строку аргументы: число строк матрицы rows, число столбцов матрицы cols, размер окна фильтра window_size, количество применений фильтра K и максимальное число потоков max_threads. Затем программа выделяет память под входную и выходную матрицы и заполняет входную матрицу случайными числами от 0 до 100 не включительно. Дальше инициируется семафор и запускается цикл, в котором применяется K раз медианный фильтр (алгоритм находится ниже). На каждой итерации строки раздаются потокам, параллельно обрабатываются и записываются в выходную матрицу, не пересекаясь, из-за чего нет гонки данных и не нужен mutex матрицы. Цикл дожидается завершения работы потоков и заканцивает итерацию.
Алгоритм медианного фильтра: для каждого элемента матрицы (r, c) собираются w^2 элементов в окрестности, которые сортируются и из них выбирается центральный, который вставляется на место элемента (r, c).

Схема взаимодействия



- ждём свободный слот для потока (sem_wait) и создаём новый поток (pthread_create) worker с его строками;
- аналогично делаем для оставшихся n-1 потоков;
- после завершения работы поток освобождает слот (sem_post)
- в конце дожидаемся завершения всех потоков (pthread_join) и переходим к следующей итерации по K.

Применённые системные примитивы

Примитив	Назначение
pthread_create	Создание потока
pthread_join	Ожидание завершения потока и освобождение его ресурсов
sem_init	Инициализация семафора
sem_wait	Ожидание свободного слота потока
sem_post	Освобождение слота

Ключевые фрагменты кода

Работа с потоками

src/main.c:

```
sem_t thread_semaphore; sem_init(&thread_semaphore,
0, max_threads);
 for (int iter = 0; iter < K; iter++) {     int total_threads = (rows <
max_threads) ? rows : max_threads;     int rows_per_thread = (rows +
total_threads - 1) / total_threads;     pthread_t *threads =
(pthread_t*)malloc(total_threads * sizeof(pthread_t));

    for (int t = 0; t < total_threads; t++) {
sem_wait(&thread_semaphore);

        ThreadArg *arg =
(ThreadArg*)malloc(sizeof(ThreadArg));         arg->thread_id =
t;         arg->start_row = t * rows_per_thread;         arg-
>end_row = (t + 1) * rows_per_thread;         if (arg->end_row
> rows) arg->end_row = rows;         arg->src = src;
arg->dst = dst;

        if (pthread_create(&threads[t], NULL, worker, arg) != 0)
{             perror("pthread_create");
sem_post(&thread_semaphore);
        }
    }

    for (int t = 0; t < total_threads; t++) {
pthread_join(threads[t], NULL);
    }
    free(threads);

    int **tmp =
src;     src = dst;
dst = tmp;
}
```

Основные процессы:

- sem_wait перед pthread_create гарантирует, что одновременно работающих потоков будет не больше max_threads;
- pthread_join в конце итерации гарантирует, что следующая итерация не прочитает данные из dst прошлой итерации;

- смена src и dst нужна, чтобы не использовать mutex и не замедлять параллельную работу до околопоследовательной.

Функция потока

src/main.c:

```
void *worker(void *arg) {
    ThreadArg *targ = (ThreadArg*)arg;

    for (int r = targ->start_row; r < targ->end_row; r++) {         for
(int c = 0; c < cols; c++) {             targ->dst[r][c] =
compute_median(targ->src, r, c, window_size);         }
    }

    sem_post(&thread_semaphore);

    free(targ);
return NULL;
}
```

Основные процессы:

- каждый поток пишет только в свои строки, поэтому гонок данных нет;
- sem_post освобождает слот для потока в семафоре.

Медианный фильтр

src/main.c

```
int compute_median(int **mat, int r, int c, int w)
{     int half = w / 2;     int count = w * w;
int *window = (int*)malloc(count * sizeof(int));
int idx = 0;

    for (int i = -half; i <= half; i++) {         for (int j =
-half; j <= half; j++) {             int nr = r + i;
int nc = c + j;             if (nr < 0 || nr >= rows || nc < 0
|| nc >= cols) {                 window[idx++] = 0;
            } else {
window[idx++] = mat[nr][nc];
            }
        }




    }
```

```
    qsort(window, count, sizeof(int), cmp_int);
int median = window[count / 2];     free(window);
    return median; }
```

Основные процессы:

- окно собирается в одномерный массив из w^2 элементов
- границы матрицы заполняются нулями
- qsort с взятием центрального элемента дают медиану

Результаты работы

Базовый сценарий

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab3/os_lab3$ build/lab3 10 10 3 1 1
Параметры: 10 x 10, окно 3, K=1, макс. потоков=1
Исходная матрица:
  16   34   38   92   91   82   85   62   61   45
  40   47   94   56   67   44   59   32   20   97
  99   56   98   33   46   64   24   51   31   44
  68   99   79   58   91   70   41   77   85    2
  74   77    1   20   33   20   65   45    5   37
  42    4   94   40   37   92   56   61   96   40
   6   16   39   37   74   83   59   15   12   96
  69   38   73   70   59    7   43   76    4   48
  13   46    4   59   38   41    4   95   55    0
  35   13   16   26    2   90    9   61   58   73

Результат после K итераций:
   0   34   38   56   56   59   44   32   32    0
  34   47   56   67   64   64   59   51   45   31
  47   79   58   67   58   59   51   41   44   20
  68   77   58   46   46   46   51   45   44    5
  42   74   58   40   40   56   61   61   45    5
   6   39   37   37   37   59   59   56   40   12
   6   39   39   59   59   59   59   56   48   12
  13   38   39   59   59   43   43   43   48    4
  13   35   38   38   41   38   43   55   58    4
   0   13   13    4   26    4    9    9   55    0
```

Демонстрация улучшения времени работы с увеличением потоков

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab3/os_lab3$ for p in 1 2 4 8 16 32 64; do    build/lab3 10000 10000 3 1 $p;   done;
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=1
15.393603
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=2
8.208222
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=4
4.370540
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=8
2.868678
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=16
1.937387
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=32
1.937385
Параметры: 10000 x 10000, окно 3, K=1, макс. потоков=64
1.928473
```

Для простоты исследования и ускорения работы вывод матриц был опущен. Видно, что с увеличением числа потоков уменьшается время работы, пока оно не становится примерно постоянным.

Диагностика через strace

С помощью трассировщика strace была проведена диагностика работы программы (команда: strace -f -e trace=clone3,futex,exit,exit_group -o trace.log build/lab3 100 100 3 1 4 2). Трасса была отфильтрована по вызовам clone3 (создание потока), futex (ожидание завершения потока), exit (завершение работы потока) и exit_group (завершение всех потоков группы) в целях исключения ненужного шума.

```
3277
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSV
SEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7041869ffce8, parent_tid=0x7041869ff990, exit_signal=0,
stack=0x7041861ff000, stack_size=0x7fff80, tls=0x7041869ff6c0} =>
{parent_tid=[3278]}, 88) = 3278
3277
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSV
SEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7041861fece8, parent_tid=0x7041861fe990, exit_signal=0,
stack=0x7041859fe000, stack_size=0x7fff80, tls=0x7041861fe6c0} =>
{parent_tid=[3279]}, 88) = 3279
3277
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSV
SEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7041859fdce8, parent_tid=0x7041859fd990, exit_signal=0,
stack=0x7041851fd000, stack_size=0x7fff80, tls=0x7041859fd6c0} =>
{parent_tid=[3280]}, 88) = 3280
3278  exit(0)                           = ? 3278  +++ exited
with 0 +++
3279  exit(0)                           = ?
3277
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSV
SEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7041851fcce8, parent_tid=0x7041851fc990, exit_signal=0,
stack=0x7041849fc000, stack_size=0x7fff80, tls=0x7041851fc6c0} <unfinished ...>
3279  +++ exited with 0 +++
3277  <... clone3 resumed> => {parent_tid=[3281]}, 88) = 3281 3280
exit(0)                           = ?
3280  +++ exited with 0 +++
3277  futex(0x7041851fcce8, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 2, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...> 3281
exit(0)                           = ?
3277  <... futex resumed>)              = 0 3281
+++ exited with 0 +++
3277  exit_group(0)                     = ?
3277  +++ exited with 0 +++
```

Здесь показывается создание потоков (вызов clone3 с флагом CLONE_THREAD) с TID 3278, 3279, 3280, 3281. По ID в начале каждой строки видно, что потоки работают параллельно.
futex(FUTEX_WAIT_BITSET) - это pthread_join, ожидающий завершение потока.

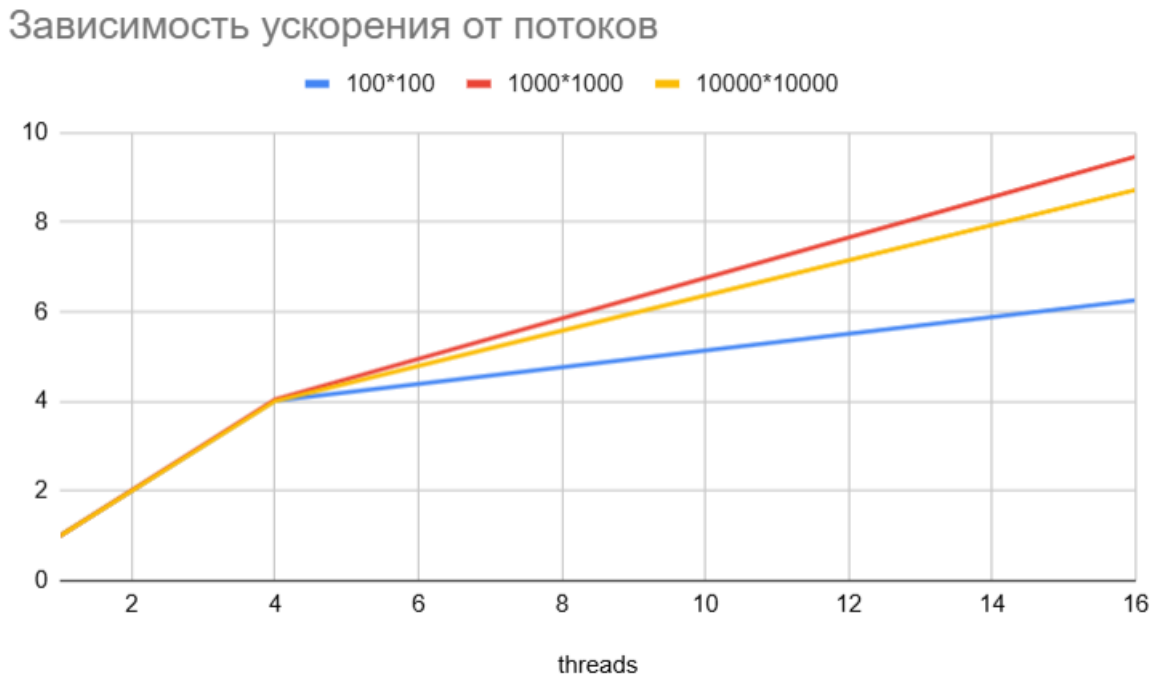
Проверка отсутствия утечек

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab3/os_lab3$ valgrind --leak-check=full --track-fds=yes build/lab3 100 100 3 1 4
==3302== Memcheck, a memory error detector
==3302== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3302== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3302== Command: build/lab3 100 100 3 1 4
==3302==
Параметры: 100 x 100, окно 3, K=1, макс. потоков=4
0.060651
==3302==
==3302== FILE DESCRIPTORS: 3 open (3 inherited) at exit.
==3302==
==3302== HEAP SUMMARY:
==3302==     in use at exit: 0 bytes in 0 blocks
==3302==   total heap usage: 10,212 allocs, 10,212 frees, 443,872 bytes allocated
==3302==
==3302== All heap blocks were freed -- no leaks are possible
==3302==
==3302== For lists of detected and suppressed errors, rerun with: -s
==3302== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Проверка отсутствия зомби-потоков

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab3/os_lab3$ ps -eo pid,ppid,nlwp,stat,cmd | grep lab3
   3718    3362    1 S+   grep --color=auto lab3
```

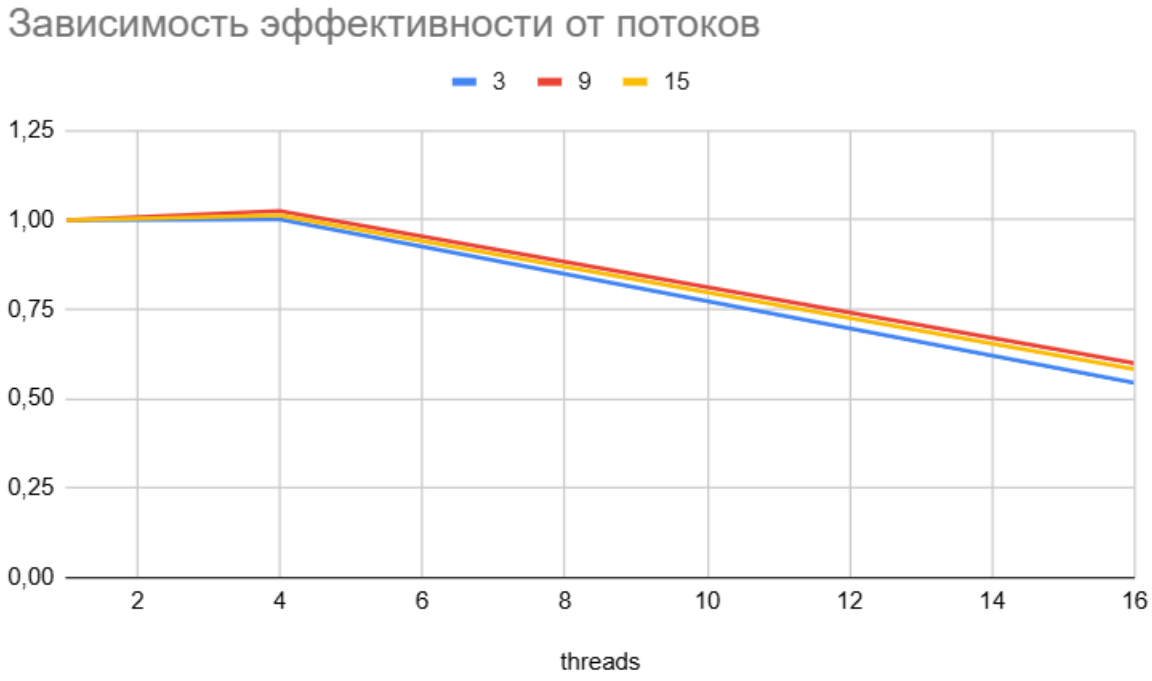
Исследование зависимости ускорения от числа потоков



Анализ результатов:

При threads = [1, 4] ускорение увеличивается около-линейно. Это означает, что алгоритм хорошо распараллеливается: строки обрабатываются независимо, гонок данных нет, накладные расходы на обслуживание потоков малы.
При threads = [4, 16] ускорение сильно падает. Причины - увеличение накладных расходов, конкурирование потоков за ресурсы.
При threads = [16, + ∞) ускорение становится становится около-константным. Причины - число потоков превышает число физических CPU (16), потоки конкурируют за ядра, кэш и TLB.
Также можно заметить, что при threads = [4, 16] ускорение увеличивается по разному на разном объёме данных: при размере 100*100 на каждый поток идёт достаточно мало работы, и накладные расходы на обслуживание потоков составляют ощутимую долю времени работы, в то время как при размере 10000*10000 накладные расходы пренебрежимо малы, но на таком объёме программа упирается в пропускную способность оперативной памяти, из-за чего разница в ускорении между 1000*1000 и 10000*10000 не такая большая, как между 100*100 и 1000*1000.

Исследование зависимости эффективности от числа потоков



Анализ результатов:
При threads = [1, 4] эффективность остаётся около-константной. Это означает, что алгоритм хорошо распараллеливается: строки

обрабатываются независимо, гонок данных нет, накладные расходы на обслуживание потоков малы.
При threads = [4, + ∞) эффективность падает. Причины - увеличение накладных расходов, конкурирование потоков за ресурсы, которых к стремлении к бесконечности начинает не хватать.

Зависимость времени от размера матрицы и окна

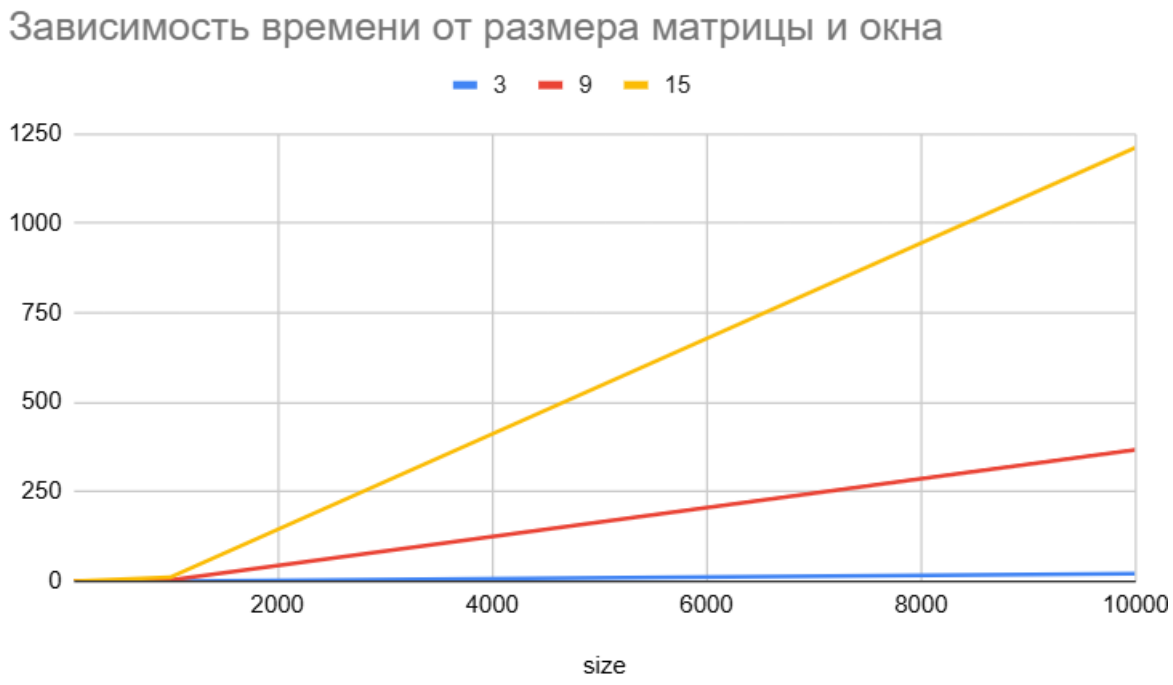


График показывает, что алгоритм быстрее работает с маленькими окнами, чем с большими. Причина в этом заключается в том, что для каждого элемента матрицы нужно отсортировать w^2 элементов в окрестности, и с ростом размера w окна нужно сортировать всё больше элементов.

Выводы

В ходе лабораторной работы:

- освоены механизмы создания потоков (pthread_create), ожидания их завершения (pthread_join) и ограничения числа одновременно работающих потоков (счётный семафор sem_wait/sem_post) в POSIXсовместимой ОС Linux;
- реализован многопоточный медианный фильтр с data parallelism по полосам строк: потоки не пересекаются по записи, поэтому гонок данных нет, и мьютекс для матрицы не требуется;
- изучена внутренняя реализация pthread через системные вызовы: clone3 с флагами CLONE_VM | CLONE_THREAD | CLONE_SIGHAND | ... создаёт поток, а futex(FUTEX_WAIT_BITSET) реализует pthread_join; exit(0) завершает поток, exit_group(0) — всю группу;
- реализована обработка системных ошибок: проверяются возвраты pthread_create, sem_init, malloc, pipe; ошибки выводятся через perror;
- обработаны краевые случаи:
    - нечётность окна — проверяется при разборе аргументов; ○ max_threads < 1 — приводится к 1; ○ max_threads > rows — уменьшается до rows (нельзя раздать больше потоков, чем строк);
    - pthread_create вернул ошибку — слот семафора освобождается через sem_post, чтобы не зависнуть;
- подтверждено отсутствие утечек памяти, файловых дескрипторов и зависаний через valgrind и /proc;
- программа проверена под strace: показано, что clone3 вызывается ровно max_threads раз на итерацию, futex реализует join, exit_group завершает процесс;
- исследовано масштабирование: ускорение растёт почти линейно до $p \approx 4$, затем насыщается из-за увеличения накладных расходов; плато - $p = 16$ (число логических CPU); при $p = 32$ прироста нет. При этом эффективность имеет плато при $p = [1, 4]$, при $p = [4, +\infty)$ эффективность постоянно падает. Также исследования показали, что алгоритм быстрее работает при маленьких окнах.

Приложения

Приложение А: ссылка на Git

https://github.com/Tr0f1mka/os_lab3